

Instituto Federal do Paraná — Campus Pinhais

E-commerce com cupons de desconto

****Aluno:**** Eros Henrique ****Disciplina:**** Testes de Software ****Professor:**** Gerson Peres
****Entrega prevista:**** 08/10/2026

1. Introdução e objetivos

Este trabalho implementa um mini e-commerce com marketplace, salespage e carrinho. O objetivo é demonstrar regras de elegibilidade, cálculo monetário em centavos e uma estratégia de testes reproduzível.

2. Módulo, requisitos e regras

O módulo aceita itens por ID e quantidade, consulta preços no backend e calcula um orçamento com um cupom por vez. As seis regras estão detalhadas em [`docs/business-rules.md`](../docs/business-rules.md). O subtotal mínimo usa todos os itens; o desconto usa apenas os elegíveis.

3. Arquitetura e tecnologias

Next.js App Router/Tailwind compõem a interface; NestJS, DTOs e Prisma 6/SQLite compõem a API; `CouponEvaluator` é a unidade de domínio. Jest testa unidade, Supertest integração e Playwright a jornada E2E. Detalhes: [`docs/architecture.md`](../docs/architecture.md).

4. Erro, defeito, falha; verificação e validação

Erro é uma decisão humana incorreta (por exemplo, confundir subtotal elegível com integral); defeito é o código ``>`` no lugar de ``>=``; falha é a resposta rejeitada para R\$ 100. Verificação examina artefatos (typecheck, testes e schema); validação observa o uso (cotação e jornada do carrinho).

5. Plano, níveis e tipos de teste

O plano, escopo, riscos e critérios estão em [`docs/test-plan.md`](../docs/test-plan.md). Testes unitários isolam todas as ramificações; integração confirma banco/HTTP; E2E confirma acessibilidade e persistência observável.

6. Particionamento, limites e tabela de decisão

As partições de subtotal são `<10000`, `=10000` e `>10000`; para SUPER20, `<20000`, `=20000`, `>20000`; quantidade válida é de 1 a 99. Valores representativos são 9999/10000/10001, 19999/20000/20001 e 0/1/99/100. A tabela SUPER20 está em [`docs/decision-table.md`](../docs/decision-table.md).

7. Casos de teste formais

Doze casos com pré-condição, passos, entradas, resultado e prioridade estão em [`docs/test-cases.md`](../docs/test-cases.md).

8. TDD e automação

Os ciclos de mínimo, elegibilidade e arredondamento são descritos em [`docs/tdd-evidence.md`](../docs/tdd-evidence.md). A suíte de domínio contém testes parametrizados, relógio fixo, exclusões, cupom temporal, total não negativo e arredondamento.

9. Integração, E2E e cobertura

Integração verifica catálogo, cenários de orçamento, manipulação de preço e validação; 5 testes foram aprovados. O teste Playwright acessa marketplace e carrinho com serviços reais (1 teste aprovado). A cobertura observada do motor foi 100% em statements/lines/functions e 96% em branches (`npm run test:coverage`).

10. Defeitos controlados

D01 (limite exclusivo) e D02 (ignorar exclusões) estão em [`docs/defect-reports.md`](../docs/defect-reports.md). São mutações temporárias e não fazem parte da implementação final.

11. Ferramentas

Jest oferece testes unitários rápidos e cobertura; Supertest exercita contratos HTTP sem substituir a aplicação; Playwright automatiza interface e fluxo real. Postman é útil para exploração manual de endpoints, enquanto k6 mede carga e latência — ambos complementares, mas não substituem os testes determinísticos do projeto.

12. Análise crítica, limitações e melhorias

O uso de centavos elimina erros de ponto flutuante e a API é a fonte canônica. O escopo não inclui autenticação, pagamento, estoque ou execução remota do GitHub Actions; como próximos passos, adicionaria testes de contrato gerados, observabilidade e cenários de concorrência.

13. Conclusão e referências

O projeto entrega um fluxo de compra local com regras auditáveis e testes automatizados. Código e comandos reproduzíveis estão no README e nos diretórios ``apps/``, ``packages/``, ``docs/`` e ``evidence/``.